

Combining Datasets: Concat and Append

Python Data Science Handbook — Chapter 3, Section 3.6

Stacking and gluing frames together — and why Pandas keeps your indices even when they collide.

1 Why combining is its own topic

Real analysis almost never starts from one tidy table. You pull last month's sales from one export, this month's from another, and you need them as a single frame. You scrape two columns of metadata from one source and three from another, all describing the same rows. The mechanical act of *gluing tables together* is so common that Pandas gives it a dedicated, deceptively simple tool: `pd.concat`.

The catch — and the reason this short section deserves real attention — is that “glue two tables together” hides a surprising amount of decision-making. Along which axis? What happens when the row labels of the two pieces *overlap*? What if the two pieces don't have the same columns? Pandas makes a specific default choice for each of these, and the defaults are not always what someone coming from SQL would expect.

Background & Context

The NumPy foundation: `np.concatenate`. Before Pandas, the array world already had a concatenation primitive. `np.concatenate` takes a *list* of arrays and joins them end-to-end along a chosen axis:

```
import numpy as np
x = np.array([[1, 2],
              [3, 4]])
np.concatenate([x, x], axis=0)    # stack rows → shape (4, 2)
np.concatenate([x, x], axis=1)    # stack cols → shape (2, 4)
```

Two things carry straight over into Pandas: (1) you pass a *list* of objects, not two positional arguments, and (2) `axis` chooses the direction of growth — `axis=0` adds rows (the array gets taller), `axis=1` adds columns (the array gets wider). The crucial *new* thing Pandas adds is that the pieces carry *labels*, and those labels must be reconciled. That reconciliation is the whole story below.

1.1 A toy-frame helper

To see the label behavior clearly, it helps to make small frames whose every cell announces where it came from. We'll use a tiny factory function throughout:

```
import pandas as pd

def make_df(cols, ind):
    """Build a small DataFrame whose cell 'C0', 'C1', ... is labeled by
    its column letter and row number, e.g. cell 'B2'."""
    data = {c: [str(c) + str(i) for i in ind] for c in cols}
```

```
return pd.DataFrame(data, ind)

make_df('ABC', range(3))
```

	A	B	C
0	A0	B0	C0
1	A1	B1	C1
2	A2	B2	C2

Every value tells you its address: B2 is column B, row 2. When we concatenate frames, we can read off exactly which original cell landed where — and, more importantly, what happened to the row *labels* on the left.

2 pd.concat: the main tool

The signature you'll use ninety percent of the time is just `pd.concat([df1, df2])`. By default it concatenates along `axis=0`, stacking the second frame's rows below the first:

```
df1 = make_df('AB', [1, 2])
df2 = make_df('AB', [3, 4])
pd.concat([df1, df2])
```

	A	B
1	A1	B1
2	A2	B2
3	A3	B3
4	A4	B4

The columns A and B matched up, the rows stacked, and the original index labels (1,2 then 3,4) were preserved. Switch to `axis=1` (equivalently `axis='columns'`) and the frames sit side by side, now aligning on the *row* index instead:

```
df3 = make_df('AB', [0, 1])
df4 = make_df('CD', [0, 1])
pd.concat([df3, df4], axis=1)
```

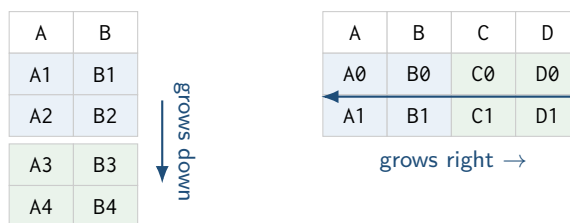
	A	B	C	D
0	A0	B0	C0	D0
1	A1	B1	C1	D1

Intuition

The mental model is a T-shaped choice. `axis=0` grows the table *downward* and lines the pieces up by matching *column* names. `axis=1` grows the table *rightward* and lines the pieces up by matching *row* index labels. Whichever axis you concatenate *along*, the *other* axis is the one Pandas aligns on. So column-wise concatenation that fails to line up rows, or row-wise

concatenation across mismatched columns, will quietly produce NaNs — not an error.

axis=0 (stack rows; match columns) **axis=1** (stack cols; match index)



3 The catch: duplicate indices survive

Here is the single most important behavioral fact in this section, and the one that separates `pd.concat` from a database table's INSERT/APPEND. **Pandas preserves index labels even when they collide.** It does *not* silently renumber the result.

```
x = make_df('AB', [0, 1])
y = make_df('AB', [0, 1])      # same index labels as x!
pd.concat([x, y])
```

```
   A  B
0  A0 B0
1  A1 B1
0  A0 B0
1  A1 B1
```

Look at the index: 0, 1, 0, 1. The label 0 now appears *twice*. The result is a perfectly valid DataFrame, but a treacherous one: `result.loc[0]` no longer returns a single row, it returns *two*, and many downstream operations that assume unique labels will behave strangely.

Key Idea

A SQL APPEND produces a table with fresh, automatically managed row identity — you never see two rows with the same primary key just because they came from different sources. `pd.concat` is more literal: it treats the index as *data you chose*, and it will not throw your labels away on your behalf. That is the right default for a library where the index often carries real meaning (dates, ids, state names). But it means *you* are responsible for deciding what should happen on overlap.

Pandas gives you three explicit remedies, depending on what you actually want.

3.1 Remedy 1: `verify_integrity=True` — refuse to overlap

If overlapping labels would be a *bug* in your pipeline, ask Pandas to police it. Setting `verify_integrity=True` makes `concat` check for duplicate labels on the concatenation axis and raise a `ValueError` if it finds any:

```
try:
```

```
pd.concat([x, y], verify_integrity=True)
except ValueError as e:
    print("ValueError:", e)
```

```
ValueError: Indexes have overlapping values: Int64Index([0, 1], dtype='
int64')
```

This is the cheap insurance option: it costs a little time (Pandas has to build the index and check uniqueness) but turns a silent data-quality problem into a loud failure. Reach for it when the indices are *supposed* to be disjoint and an overlap means something upstream went wrong.

3.2 Remedy 2: `ignore_index=True` — throw the labels away

Sometimes the original index is meaningless (it’s just “row 0, row 1…” from each source) and all you want is a clean, continuous numbering for the combined frame. `ignore_index=True` discards both input indices and builds a fresh `RangeIndex`:

```
pd.concat([x, y], ignore_index=True)
```

	A	B
0	A0	B0
1	A1	B1
2	A0	B0
3	A1	B1

Now the index reads 0, 1, 2, 3: unique, contiguous, and forgettable. This is the right call whenever the index was incidental record-keeping rather than meaningful labels.

3.3 Remedy 3: `keys=[...]` — remember the source

The third option *keeps* the original indices but disambiguates them by adding an outer level that records which frame each row came from. The result is a hierarchically indexed (`MultiIndex`) `DataFrame`:

```
pd.concat([x, y], keys=['first', 'second'])
```

		A	B
first	0	A0	B0
	1	A1	B1
second	0	A0	B0
	1	A1	B1

Now `result.loc['first']` cleanly recovers the original `x`, and the source label travels with the data. This is the most *informative* remedy: it doesn’t discard anything, it adds structure. It’s the natural choice when the provenance matters — “these rows are January, those are February” — because you can later group, slice, or unstack on that outer level.

Intuition

Read the three remedies as three answers to the question “what do these colliding labels *mean*?” If they mean *a mistake*, use `verify_integrity`. If they mean *nothing*, use `ignore_index`. If they mean *two different sources I want to keep track of*, use `keys`. There is no universal right answer — only the one that matches your data’s semantics.

4 Concatenation with joins: mismatched columns

So far the frames shared columns. What happens when they don’t? Suppose one frame has columns A,B,C and another has B,C,D, and we stack them row-wise:

```
df5 = make_df('ABC', [1, 2])
df6 = make_df('BCD', [3, 4])
pd.concat([df5, df6])          # default join='outer'
```

	A	B	C	D
1	A1	B1	C1	NaN
2	A2	B2	C2	NaN
3	NaN	B3	C3	D3
4	NaN	B4	C4	D4

The default `join='outer'` takes the *union* of the column sets ($\{A, B, C\} \cup \{B, C, D\} = \{A, B, C, D\}$). Where a frame lacked a column, Pandas fills NaN. This is the safe, lossless default: nothing is dropped, but you may inherit holes.

Switch to `join='inner'` to keep only the columns the frames have in *common* — the *intersection*:

```
pd.concat([df5, df6], join='inner')
```

	B	C
1	B1	C1
2	B2	C2
3	B3	C3
4	B4	C4

Only $\{A, B, C\} \cap \{B, C, D\} = \{B, C\}$ survive, and there are no NaNs because every retained column exists in both frames.

outer (union of columns)

A	B	C	D
A1	B1	C1	NaN
A2	B2	C2	NaN
NaN	B3	C3	D3
NaN	B4	C4	D4

inner (intersection of columns)

B	C
B1	C1
B2	C2
B3	C3
B4	C4

only shared columns kept; no holes

Common Pitfall

The old `join_axes` parameter is gone. Older tutorials (and earlier printings of the Handbook) show a `join_axes=[...]` argument that let you pin the result's columns to a specific frame's column list. That parameter was *removed* in pandas 1.0. If you need the result to carry exactly some chosen set of columns, do the concat first and then `.reindex(columns=...)`:

```
out = pd.concat([df5, df6])
out = out.reindex(columns=df5.columns)    # keep A, B, C in that order
```

Same effect, explicit and future-proof.

5 DataFrame.append: a deprecated shortcut you will still see

For years, Pandas offered a convenience method, `df1.append(df2)`, that was essentially `pd.concat([df1, df2])` for the common “stack one frame under another” case. You will encounter it constantly in older notebooks, Stack Overflow answers, and blog posts:

```
# Historical one-liner (do NOT use in new code):
df1.append(df2)                # was equivalent to pd.concat([df1, df2])
```

Common Pitfall

DataFrame.append is deprecated and has been removed. It was deprecated in pandas 1.4 (early 2022) and *removed entirely* in pandas 2.0 (April 2023). Calling it on a modern install raises `AttributeError: 'DataFrame' object has no attribute 'append'`. There were two good reasons to retire it:

- **It was a performance trap.** Unlike Python's `list.append`, the Pandas method did *not* mutate in place — it built and returned a brand-new frame every call. Writing **for** chunk **in** chunks: `df = df.append(chunk)` therefore did $O(n^2)$ copying. The idiomatic replacement, `pd.concat([...])` over a *list* of frames built once, is far faster.
- **It was redundant.** Everything `append` did, `concat` does, with more control (axes, joins, keys).

Migration is mechanical: replace `a.append(b)` with `pd.concat([a, b])`, and replace any accumulation loop with “collect frames in a list, `concat` once at the end.”

Python Refresher

The name collision is worth pausing on, because it bites newcomers. Python's built-in `list.append` *mutates* the list in place and returns `None`. The late Pandas `DataFrame.append` did the *opposite*: it returned a new frame and left the original untouched. Same word, opposite contract. The cleanest way to avoid the confusion entirely is to forget the Pandas method exists and reach for `pd.concat` — whose list-of-frames argument also nudges you toward the efficient “build the list, `concat` once” pattern.

Section recap

- `pd.concat(...)` takes a *list* of objects; `axis=0` (default) stacks rows and aligns on columns, `axis=1` stacks columns and aligns on the row index.
- Unlike a database append, `concat` **preserves duplicate index labels** rather than renumbering — a frequent source of surprise.
- Three remedies for colliding labels: `verify_integrity=True` (raise on overlap), `ignore_index=True` (fresh `RangeIndex`), `keys=...` (add a `MultiIndex` source level).
- With mismatched columns, `join='outer'` (default) takes the *union* and fills NaN; `join='inner'` keeps only the *intersection*. The old `join_axes` is gone — use `reindex` instead.
- `DataFrame.append` is **removed in pandas 2.0**; always use `pd.concat(...)`, and accumulate by collecting frames in a list and concatenating once.